

Uniswap Margin Trading Audit



Uniswap

August 24, 2026

This document has been prepared solely for the named client and it may not be relied upon by any third parties. The content in this document is provided for informational purposes only and does not constitute professional advice of any kind. Security assessments are limited by factors such as scope, time, and techniques used and therefore this document does not constitute a certification, guarantee, or warranty regarding the proper functioning of any system or the absence of security vulnerabilities.

Copyright © 2026 Zeppelin Group Ltd.

Table of Contents

Table of Contents	2
Summary	4
Scope	5
System Overview	7
Security Model and Trust Assumptions	8
Medium Severity	10
M-01 The Routed-Swap Fill Assertion Measures an Absolute Balance Rather Than the Amount Delivered	10
Low Severity	11
L-01 Cached Aave Protocol Data Provider Cannot Be Replaced	11
L-02 The Morpho Repay Encoder Reverts on Two Boundary Amounts	12
L-03 A Dust aToken Transfer Permanently Prevents an Account's Collateral from Backing a Borrow	13
L-04 Governance Cannot Stop Routing to an Individual Market	14
L-05 The Aave V4 Reserve Identifier Binding Is Validated Only at Registration	15
L-06 ROUTE_SWAP Dispatches Position Swaps to an Unvalidated Caller-Supplied Address	16
L-07 Curated Entry Points Roll Back Completed Mutations When an Event-Only Read Reverts	16
L-08 Retiring a Market Breaks the Router Exit Path for Existing Positions in That Pair	17
L-09 Full-Close Event Hardcodes a Position State the Adapters Cannot Guarantee	18
L-10 ROUTE_SWAP Leaves a Spendable Permit2 Allowance on the Router	20
L-11 An Increase Supplies the Account's Entire Collateral Balance With No Way to Opt Out	21
L-12 Full Close of a Debt-Free Empty Position Is a Silent No-Op	22
L-13 Re-Pointing a Registered Pair Orphans Every Position Already Opened Under It	23
L-14 Repay Amount Clamping Is Inconsistent Across Adapters and Misreported	24
L-15 A Drifted Compound V3 Position Rejects Small Partial Decreases	25
L-16 CompoundV3LendingAdapter Values Debt With a Superseded Comet Price Feed	26
Notes & Additional Information	28
N-01 Misconfigured Morpho Markets Can Be Registered as Routable	28
N-02 Morpho Position Views Revert on Reads the Result Does Not Depend On	29
N-03 Pending Ownership Handoffs Cannot Be Canceled	30
N-04 Shadowed State Variable	30
N-05 CollateralWithdrawn Always Reports a Zero Amount on Three of the Four Venues	31
N-06 Unused Named Return Variables	31
N-07 Unused Code	32
N-08 Inconsistent Interface Convention in Storage Types	32
N-09 An Unsupported Action Opcode Is Reported as a Missing Active Account	33
N-10 Zero Is Overloaded as a Control Signal in Amount and Health-Bound Fields	33

N-11 Sweeping Native Currency From an Account to the Manager Always Reverts	34
N-12 Functions Updating State Without Event Emissions	35
N-13 A Comet Base Supply Position Is Not Reachable Through the Router	35
N-14 Priced Reads Revert if a Comet Collateral Asset Is Removed	36
N-15 Incomplete and Inaccurate Documentation	37
N-16 Market Registration Validates the Wrong Reserve Property on Both Sides of the Pair	38
Conclusion	40
Appendix	42
Issue Classification	42

Summary

Type	DeFi	Total Issues	33 (14 resolved, 7 partially resolved)
Timeline	From 2026-07-28 To 2026-08-12	Critical Severity Issues	0 (0 resolved)
Languages	Solidity	High Severity Issues	0 (0 resolved)
		Medium Severity Issues	1 (1 resolved)
		Low Severity Issues	16 (6 resolved, 2 partially resolved)
		Notes & Additional Information	16 (7 resolved, 5 partially resolved)
		Client Reported Issues	0 (0 resolved)

Scope

OpenZeppelin performed an audit of the [Uniswap/v4-periphery](#) repository at commit [e9b50f5](#).

In scope were the following files:

```
src
├─ AaveLendingAdapter.sol
├─ AaveV4LendingAdapter.sol
├─ CompoundV3LendingAdapter.sol
├─ MarginAccount.sol
├─ MarginAccountFactory.sol
├─ MarginRouter.sol
├─ MorphoLendingAdapter.sol
├─ V4Router.sol
├─ base
│   └─ OwnableAdapter.sol
├─ interfaces
│   ├── ILendingAdapter.sol
│   ├── IMarginAccount.sol
│   ├── IMarginRouter.sol
│   ├── IPermit2Forwarder.sol
│   └─ external
│       ├── aave-v4
│       │   ├── IAaveOracle.sol
│       │   └─ ISpoke.sol
│       ├── aave
│       │   ├── IAaveOracle.sol
│       │   ├── IPool.sol
│       │   ├── IPoolAddressesProvider.sol
│       │   └─ IPoolDataProvider.sol
│       └─ compound-v3
│           └─ IComet.sol
├─ libraries
│   ├── MarginActions.sol
│   └─ MarginCallldataDecoder.sol
└─ types
    ├── Ltv.sol
    ├── Market.sol
    ├── MarketAllowlist.sol
    ├── MarketRegistry.sol
    ├── Owner.sol
    └─ PositionData.sol
```

The files introduced by [pull request #563](#) were audited in full. Two of the files listed above, namely [V4Router.sol](#) and [IPermit2Forwarder.sol](#), are pre-existing files that the pull request only modifies. Those two were reviewed with respect to the changes they received and

were not audited in full, so any observation about them is limited to the modified lines and their immediate interaction with the margin trading contracts.

System Overview

The margin trading periphery lets a user open leveraged spot exposure in a single transaction by composing a Uniswap v4 swap with a borrow against an external lending protocol. A position is described only by a `(collateral, debt)` token pair, and that pairing alone sets the direction of the trade: the holder is long the collateral and short the debt. Leverage is assembled inside one `PoolManager` unlock using v4 flash accounting, which allows the debt token to be swapped for collateral before it has been borrowed. The periphery contains no liquidation machinery of its own: a position that becomes unhealthy is liquidated by the lending venue directly against the account, out of band, and nothing in the margin trading contracts observes or signals that event.

MarginRouter: The entry point and the orchestrator of every flow. It exposes three curated operations, namely opening or increasing a position, reducing or fully closing one, and adding collateral without changing debt. Each curated operation builds a fixed sequence of actions and runs it inside a single unlock: route the swap, assert that the bought amount arrived, supply the collateral, draw the debt, and settle. The router additionally exposes a general-purpose `execute` entry point that interprets an arbitrary caller-supplied plan drawn from the same action set, combining v4 routing primitives with account-scoped lending actions. The curated operations enforce slippage and health bounds, while `execute` enforces none and its documentation places on the plan author the obligation to net the router to zero and to append health assertions.

MarginAccount: A per-user container deployed as a clone with immutable arguments, one per `(owner, subId)` pair, at an address derived deterministically from those values. The account is itself the supplier and borrower at the lending protocol, so it acts on its own behalf and never requires delegated authorization. Owner and manager are baked into the clone bytecode, which means ownership is fixed at deployment and cannot be transferred. Every fund-moving primitive is callable only by the manager, which is the router, or by the owner, and recipients of withdrawals, borrows, and sweeps are constrained to those same two addresses. The owner additionally holds an escape hatch that forwards arbitrary calldata to the lending protocol, so a position can be managed without the router.

Lending adapters: One singleton adapter per venue, covering Morpho Blue, the Aave v3 Pool, a single Aave v4 Spoke, and a single Compound v3 Comet. An adapter holds no funds and moves no tokens. It is an encoder: for each of the four lending primitives it returns the target,

value, and calldata that the account then executes as itself, and it exposes the reads the router uses for health and event data. Each adapter keeps a governed table of routable token pairs, so only curated markets are reachable. Because the account does not decode the calldata an adapter returns, the adapter is trusted to name the lending protocol as the call target, to pass a zero value, and to act on the account's own balances.

Swap routing: Position swaps are routed through the Universal Router rather than a single v4 pool, so liquidity can be sourced across v2, v3, and v4. Since the Universal Router settles its own swap, the router wraps the call in a flash-take envelope: it takes the input from the `PoolManager`, grants a scoped allowance so the Universal Router can pull only what the swap costs, runs the caller-built route, and settles the unspent remainder. The caller supplies both the route and the Universal Router address on every call, and the router requires only that the route deliver the bought output to the account and stay within the supplied input cap.

Security Model and Trust Assumptions

Governance (`MarginRouter`): Curates the set of lending adapters that the exposure-increasing flows may use, and hands its own role over through a two-step proposal and acceptance. The allowlist gates only supplying collateral and borrowing. Repaying, withdrawing collateral, and sweeping are left ungated so that a position remains exitable after an adapter is retired. Governance is trusted not to allowlist an adapter that would redirect a caller's equity, since an adapter reaches the account through the router's manager privilege.

Adapter owners: Each adapter has its own owner, independent of router governance, which registers and retires routable token pairs and likewise hands over its role in two steps. An adapter owner is trusted to register only markets whose configuration the protocol will actually serve, because registration validation differs by venue and a registered pair is reported as routable regardless of whether it can be opened. Retiring or replacing a pair affects every position already opened under it, so owners are trusted to consider existing positions before curating a live market.

Position owners: Hold full authority over their own accounts, both through the router and through the account's owner-only escape hatch. An `execute` plan can only ever reach accounts derived from the authenticated caller, so a plan cannot touch another user's position. Signing or submitting a plan built elsewhere is equivalent to handing over the sub-account,

because the router is the account manager and no token approval is required. Position owners are therefore trusted to build their own plans, and integrators are expected to construct that calldata rather than accept it from a third party.

Integrated lending protocols: The system depends on Morpho Blue, Aave v3, Aave v4, and Compound v3 for its correct operation, and this is the most consequential trust boundary in the design. Each venue is the sole authority on whether a position is solvent, on the price used to value it, and on the accounting that determines what a repayment or withdrawal actually settles. The adapters do not reimplement those decisions, so the router's own health assertions are advisory bounds layered on top of the venue's enforcement rather than a substitute for it. Position reads, and therefore the amounts the curated flows derive from them, resolve through venue price feeds, interest rate models, and reserve configuration, all of which are controlled by the respective protocol's governance and can change independently of this system. Adapters bind to those venues at deployment and are neither upgradeable nor equipped with setters, so a venue reconfiguring or upgrading a component an adapter depends on can change what the router reports or prevent flows from completing. Every venue is likewise trusted to hold custody of supplied collateral and to honor its own solvency and liquidation rules.

Universal Router and route builders: The Universal Router address and the route it executes are supplied by the caller on every position swap, and are not restricted to a known deployment. The dispatch is contained only by its shape: the router calls whatever address it is given through the fixed `execute(bytes,bytes[],uint256)` selector and validates nothing else about the target. Because the router is the manager baked into every account it deploys, and therefore a privileged caller throughout the system, this containment rests on the invariant that no contract which trusts `MarginRouter` exposes a colliding selector, and any future contract that grants the router authority must be reviewed against it. Beyond that, the router constrains the outcome only by capping the input it makes available and by asserting that the bought output reached the account, so callers are trusted to point at a Universal Router deployment they intend to use and to build a route that behaves as the surrounding flow assumes.

Tokens and markets: Curated markets are assumed to hold standard ERC-20 tokens. Fee-on-transfer and rebasing tokens are out of scope, and the netting the curated flows rely on holds only under that assumption. Because account addresses are derivable before deployment, any party can send tokens to an account or to the router in advance, so balances held outside the lending protocol are not assumed to belong to the operation in progress.

Medium Severity

M-01 The Routed-Swap Fill Assertion Measures an Absolute Balance Rather Than the Amount Delivered

Both curated position flows delegate their swap to a caller-built Universal Router route and do not trust the route's internals (see [_routeSwap](#)). The guarantee that the route actually delivered what was requested rests entirely on the [ASSERT_ACCOUNT_BALANCE](#) action. An increase [encodes its threshold as the contributed equity plus collateralToBuy](#), and a decrease [encodes the target debt](#). The corresponding [IncompleteFill](#) error documents the increase as all-or-nothing, and even documents its [received](#) parameter as "the collateral amount the swap actually delivered."

However, the [_assertAccountBalance](#) function compares the [account's absolute token balance](#) against that threshold rather than the increase across the swap. Any collateral the account already holds therefore relaxes the assertion by exactly that amount, and the documented all-or-nothing property does not hold. A pre-existing balance of the requested size disables the check entirely. Such a balance is not unusual. It arises when equity is transferred directly to the deterministic account address and the call is made with a zero [equity](#) field, which is the pattern used by the in-tree integration harness and the natural pattern for a contract integrator avoiding the Permit2 pull. It also arises from funds staged into the account by an earlier [execute](#) plan, from a token that is the collateral of one market and the debt of another under the same [subId](#), which Morpho's isolated markets permit, and from an unsolicited transfer by any third party, since the account address is public through [accountOf](#).

The consequence is that the router accepts a position built on a short fill without reverting. Neither [maxDebtIn](#) nor [maxCollateralIn](#) compensates, because those bound the input consumed rather than the output delivered, and the shortfall is silently made up from the owner's idle balance, which is then [supplied into the lending market](#). The optional [ASSERT_HEALTH](#) check does not compensate either: the same idle collateral that masks the shortfall keeps the resulting LTV low. A route that consumes its full input allowance while delivering less than requested is therefore not detected.

Consider recording the account's balance of the relevant currency before the unlock begins, i.e., before equity is transferred in, and asserting the increase across the swap rather than the resulting balance. `_routeSwap` already establishes this pattern for the router's own balance with its `balanceBeforeTake_snapshot`, so that only this call's flash-take is settled and a donated balance cannot subsidize the position; the fill assertion needs the equivalent treatment for the account balance. Thorough unit testing of the increase, the partial decrease, and the full close is essential, since all three encode this assertion.

Update: Resolved at commit [7952306](#).

Low Severity

L-01 Cached Aave Protocol Data Provider Cannot Be Replaced

The `AaveLendingAdapter` `constructor` resolves the Aave Pool and the protocol data provider from the addresses provider once and stores both as immutables, and the [contract documentation](#) justifies this by asserting that both "are upgradeable proxies whose addresses are stable across Aave upgrades". This is true for the Pool but not for the data provider. In Aave's `PoolAddressesProvider`, `setPoolImpl` upgrades an admin proxy in place so that `getPool` keeps returning the same address, whereas `setPoolDataProvider` overwrites the `DATA_PROVIDER` entry with a plain, non-proxied address. Aave has redeployed the protocol data provider several times in practice, and each redeployment changes the address that `getPoolDataProvider` returns.

Because the adapter is neither upgradeable nor equipped with a setter, its cached `dataProvider` can never be reointed. A redeployment is not immediately harmful, because the deregistered provider remains deployed and keeps reading live from the Pool proxy. The risk materializes once a later Pool implementation upgrade changes the reserve-data layout that the stale provider decodes, at which point its reads either revert or return the wrong reserve token addresses. This stale provider backs `maxLtvWad`, `setMarket`, and the `_reserveTokens` lookup behind `positionOf` and `describePosition`.

A revert from any of these reads causes `decreasePosition` to revert on its opening `positionOf` call, so the curated close and decrease flows become unavailable. Positions then remain exitable only through the ungated repay and withdraw actions, whose encoders never read the data provider. If the reads instead return a wrong variable debt token,

`positionOf` reports zero debt while the position is still open in Aave, and the [full-close branch](#) of `decreasePosition` treats a fully indebted position as debt-free.

Consider caching only the addresses provider and resolving the data provider on each use, or adding an owner-gated setter, and correcting the documentation accordingly.

Update: Resolved at commit [ac3071f](#).

L-02 The Morpho Repay Encoder Reverts on Two Boundary Amounts

The [encodeRepay function](#) of `MorphoLendingAdapter` encodes an [asset-denominated repayment](#) for an explicit amount and a [share-based repayment](#) for the `type(uint256).max` sentinel. Both forms produce a call that Morpho Blue rejects at a boundary the adapter does not guard.

The first boundary is the amount the adapter itself reports. `positionOf` and [describePosition](#) derive the outstanding debt through `MorphoBalancesLib.expectedBorrowAssets`, which converts borrow shares to assets rounding up. Morpho converts an asset-denominated repayment back to shares rounding down, so the reported value resolves to more shares than the account holds and the borrow-share subtraction underflows with a bare arithmetic panic. The overshoot never exceeds one asset unit, but it is non-zero for effectively every share balance on a market whose accrued assets-to-shares ratio does not divide exactly, which is the ordinary case.

The second boundary is a debt-free position. The sentinel path reads the account's borrow-share balance and encodes a repayment with both the asset and share arguments set to zero, which Morpho rejects because it requires exactly one of the two to be zero. An integrator composing a generic exit plan, in which a repayment precedes a collateral withdrawal, therefore cannot apply that plan to a position funded only through [addCollateral](#), repaid out of band, or previously liquidated.

No funds are at risk in either case, since both failures are atomic reverts reachable only by the position owner. However, the documentation steers integrators toward both paths, presenting the reported debt as an accurate snapshot of current obligations and the sentinel as the reliable way to repay in full. The behavior is furthermore inconsistent across the adapter family, because the Compound v3 adapter clamps the requested amount to the outstanding borrow and Aave v3 clamps internally, so the value reported by `positionOf` is a valid repayment input for every adapter except this one.

Consider clamping the amount within `encodeRepay` so that a value at or above the reported debt selects the share-based path, and treating a zero borrow-share balance as a no-op rather than encoding a call that cannot succeed.

Update: Resolved at commit [654e6bd](#).

L-03 A Dust aToken Transfer Permanently Prevents an Account's Collateral from Backing a Borrow

The `AaveLendingAdapter.encodeSupplyCollateral` function encodes only a call to `IPool.supply` and relies on Aave to enable the supplied reserve as collateral automatically. On the `aave-v3-origin` revision currently deployed on mainnet (`POOL_REVISION() == 11`), `executeSupply` sets the `usingAsCollateral` flag only when the supply is the account's first for that reserve, and `executeFinalizeTransfer` no longer enables collateral for the recipient of an aToken transfer. As a result, any earlier aToken transfer that leaves the account with a non-zero scaled balance makes every later supply a non-first supply, so the flag is never set. The `AaveV4LendingAdapter` batches an explicit `setUsingAsCollateral` into every supply and is therefore not affected.

Because `accountOf` derives an account address deterministically from `(owner, manager, subId)` whether or not it is deployed, and an ERC-20 transfer to an undeclared address succeeds, an attacker can send a victim's account a dust amount of aTokens in advance for the cost of the dust plus gas. The victim's subsequent `increasePosition` then supplies collateral that is never flagged, the borrow leg finds no backing, and the transaction reverts. The same failure occurs without any attacker whenever Aave sets a reserve's `ltv` to zero while keeping its `liquidationThreshold` non-zero, since Aave's auto-enable heuristic also requires a non-zero `ltv`.

No funds are placed at risk, because the increase reverts atomically and any unflagged collateral remains withdrawable. However, the condition is a persistent denial of service on opening a position for the affected `(owner, subId, reserve)` triple, and a fresh `subId` can be poisoned just as cheaply. `addCollateral` still succeeds and parks a deposit that backs nothing, while `describePosition` reports it as usable collateral. Recovery requires calling `setUserUseReserveAsCollateral` through the owner-only `execute`, which no interface exposes directly.

Consider having the adapter encode an explicit `setUserUseReserveAsCollateral(collateral, true)` sequenced after the supply.

The call is idempotent, so re-emitting it on every supply is safe, leaves existing positions untouched, and self-heals any already-poisoned account.

Update: Resolved at commit [bb92495](#).

L-04 Governance Cannot Stop Routing to an Individual Market

The `setMarket` function of `MorphoLendingAdapter` writes a Morpho `MarketParams` into the adapter's routing table, and no removal path exists. Both `resolve` and `isSupported` treat an entry as unset only when its collateral and loan tokens are both zero, while `register` derives its storage key from those same addresses. No input can therefore clear an occupied slot, and `isSupportedMarket` never returns `false` again once a pair is registered.

Governance can consequently re-point a pair at a different Morpho market, but it cannot stop the adapter from routing that pair at all. Replacement is not a substitute for revocation, because it presupposes that an acceptable alternative market exists. The only remaining lever is removing the entire adapter from the `MarginRouter` allowlist, which disables every market the adapter serves rather than the one that requires attention. There may be several reasons to retire a single market, such as utilization that becomes too high or a bug discovered in a particular market's configuration. The three sibling adapters all accept a `bool allowed` argument and can revoke a pair.

Consider adding a revocation path that mirrors the sibling adapters, either an `unregister` function that deletes the entry or an enabled flag consulted by `isSupported` and `resolve`.

Update: Acknowledged, not resolved. The team stated:

Confirmed. The Morpho registry registers/replaces but does not revoke a single pair (the sibling adapters can via `allowed=false`). Governance can still stop new exposure to a Morpho market by re-pointing it or by disallowing the adapter wholesale, and existing positions remain exitable via the escape hatch. We treat per-market revocation as a future governance-ergonomics enhancement rather than a fix, and document that stopping a single Morpho market today requires adapter-level disallow or re-point.

L-05 The Aave V4 Reserve Identifier Binding Is Validated Only at Registration

`AaveV4LendingAdapter` keys each market to a pair of per-Spoke reserve indices rather than to asset addresses. `setMarket` validates at registration that each reserve's `underlying` matches the currency being registered and that both reserves share a Hub, but `_resolveRoute` re-checks only the `registered` flag. The binding is therefore never reconfirmed at use time, even though the adapter already declares the `ReserveMismatch` error that would do so.

Because `setMarket` overwrites an occupied route without an occupancy check, the owner can re-register a live `(collateral, debt)` pair and silently repoint the reads and encodes of every position already opened under it. The registration-time check still forces the new reserve ids to a matching `underlying`, so a repoint can only move between two reserves that share an underlying. Where the Spoke lists such a pair, a full close would encode a withdrawal from a reserve in which the account holds no shares, the Spoke would return zero, and `MarginAccount.withdrawCollateral` would measure a zero collateral delta and forward nothing while emitting a successful close, leaving the collateral stranded under the original reserve id.

The severity is Low because the more damaging variant, in which a live Spoke repoints a reserve id to a different underlying so that a close delivers the wrong token, is not realizable against Aave v4's append-only reserves, and the realizable variant above requires a deliberate owner action. The binding is nonetheless the only external reference in the deploy path that is not re-derived at call time: the reserve ids are hardcoded as `bare constants`, whereas the Morpho adapter re-derives its market id on every call.

Consider storing the expected collateral and debt underlyings in `V4MarketRoute` at registration and comparing them against the reserve on every resolve, which adds no external call, or re-reading `spoke.getReserve(...).underlying` inside `_resolveRoute` and reverting with the existing `ReserveMismatch` error. In addition, consider requiring an explicit replace flag before `setMarket` may overwrite an occupied route.

Update: Acknowledged, not resolved. The team stated:

The binding can drift only via a deliberate owner re-registration to mismatched reserve ids, and the severe wrong-token variant is unrealizable (a debt-bearing close reverts rather than mis-routing). We document that reserve-id bindings must match the Spoke's live layout at registration rather than adding a per-operation reserve-underlying re-validation read.

L-06 ROUTE_SWAP Dispatches Position Swaps to an Unvalidated Caller-Supplied Address

The `ROUTE_SWAP` action routes a position swap through the Universal Router so that liquidity can be sourced across Uniswap v2, v3, and v4. The Universal Router is supplied per call rather than fixed at deployment, in `IncreaseParams` and `DecreaseParams` for the curated flows and as the first field of the action parameters for an `execute` plan. This is a deliberate design decision, documented so that callers may adopt a newer Universal Router deployment without redeploying the margin router.

However, the `_routeSwap` function validates only that the supplied address is not the zero address. There is no allowlist, no code-size check, and no verification that the target is a Universal Router. The `dispatch to the supplied address` therefore permits any account to make `MarginRouter` issue a call to any address, with fully caller-controlled arguments. The sole containment is the fixed `execute(bytes,bytes[],uint256)` function selector, so the safety of the flow rests on an implicit and undocumented invariant, namely that no contract which trusts `MarginRouter` exposes a colliding selector. That invariant is load-bearing, because `MarginRouter` is the `manager` baked into every `MarginAccount` it deploys and is therefore a privileged caller throughout the system. Furthermore, the router `grants the supplied address a Permit2 allowance` over its own balance before issuing the call, so naming an arbitrary address is not merely a call primitive but also an approval primitive.

Consider maintaining a governance-curated allowlist of permitted Universal Router addresses and rejecting any address outside it. The allowlist would preserve the ability to adopt a newer deployment while removing the arbitrary-callee and arbitrary-spender surface. Should the unrestricted per-call parameter be retained, it is advisable to document the selector-collision invariant explicitly, so that any future contract which trusts the router is reviewed against it.

Update: Acknowledged, not resolved. The team stated:

This is a design decision intended to enable future functionality in currently unreleased versions of the UniversalRouter. Callers are expected to provide a valid UR address.

L-07 Curated Entry Points Roll Back Completed Mutations When an Event-Only Read Reverts

Each curated entry point finishes by reading the resulting position through the adapter:

`increasePosition`, the `partial branch of decreasePosition`, and `addCollateral` all call `describePosition` once their state changes are complete. In all three the returned

`PositionData` is used for nothing other than populating the emitted event. The router already treats that same call as non-essential elsewhere, since `_emitPosition` wraps it in `try/catch` so that a failing read is swallowed rather than reverting the action it describes.

However, the curated reads are unguarded, so a revert in `describePosition` bubbles up and reverts the whole transaction after the position mutation has already succeeded.

`describePosition` is not guaranteed to be non-reverting, because every adapter resolves it through the venue's own `price source` and through a `market that must still be routable`, and either dependency can fail outside the router's control. The clearest consequence is on `addCollateral`, which performs no unlock and no health assertion, so a purely risk-reducing deposit cannot complete for as long as the read fails. The same applies to `increasePosition` whenever `maxLtvAfter` is zero, because that bound is optional there and nothing else fails first. A partial decrease is unaffected in practice, because its `mandatory maxLtvAfter` makes `ASSERT_HEALTH` consult the same price source inside the unlock and revert earlier.

Consider wrapping the three curated reads in `try/catch`, mirroring `_emitPosition`, and emitting the event with the derived fields omitted when the read fails, so that a completed mutation is never rolled back by an auxiliary read whose only purpose is observability.

Update: Resolved at commit [3d03ca5](#).

L-08 Retiring a Market Breaks the Router Exit Path for Existing Positions in That Pair

The router deliberately leaves the exit path ungated so a position can always be unwound through it, as stated by the comment on `_withdrawCollateral` and confirmed by `_requireAllowedAdapter` applying only to the exposure-increasing paths. That asymmetry is undone one layer down, because `decreasePosition` reads the position through the adapter first, and every adapter gates all of its reads and encoders, including `positionOf`, `encodeRepay`, and `encodeWithdrawCollateral`, on its governed market table: `_requireSupportedMarket` on `AaveLendingAdapter` and `likewise` on `CompoundV3LendingAdapter`, `_resolveRoute` on `AaveV4LendingAdapter`, and `resolve` on `MorphoLendingAdapter`.

Once governance retires a deprecated or mispriced market via `setMarket(collateral, debt, false)` on the Aave or Compound adapters, `decreasePosition` reverts with `MarketNotSupported` for every existing position in that pair, and the reads revert as well, so those positions become at once unclosable and unobservable through the router.

`MorphoLendingAdapter` cannot delist, as its `setMarket` only [registers or replaces](#) the pair's canonical market. A replacement, which occurs whenever a new oracle, IRM, or LLTV yields a new Morpho id, re-points every read and exit encoder at the new market, where the account holds nothing. Positions in the old market therefore become invisible, and a full-close `decreasePosition` silently succeeds as a no-op through the [debt-free branch](#) while collateral and debt sit untouched.

The positions are not trapped, since `sweep` is not market-gated and `repay` accepts a caller-chosen adapter [by design](#). What is lost is the atomic, self-funding unwind, since the fallbacks require the owner to front external capital or to deploy and register an adapter of their own. The trigger is a routine governance action that affects every position in the pair at once.

Consider gating only `encodeSupplyCollateral` and `encodeBorrow` on the market table and leaving the repay, withdraw, and read functions ungated, so the asymmetry the router implements is preserved by the layer it delegates to. Note that on `AaveV4LendingAdapter` this also requires that `setMarket` stop [deleting the route](#) and clear only the `registered` flag, and on `MorphoLendingAdapter` that replaced routes remain resolvable for those functions, for instance by resolving exits against the market where the account actually holds a position, so retiring a route only redirects new exposure.

Update: Acknowledged, not resolved. The team stated:

Confirmed and correctly bounded. After a de-registration the curated close/decrease and even hand-built router repay/withdraw revert (all adapter encoders gate on market support), but positions are not trapped: the owner exits directly through `MarginAccount.execute` against the lending protocol. Because de-registration is a deliberate governance action, we treat "do not de-register a pair with open positions; if you must, users exit via the account escape hatch" as operational guidance we document, rather than ungating the exit encoders (which is venue-dependent and awkward for Morpho, whose `MarketParams` live in the registry).

L-09 Full-Close Event Hardcodes a Position State the Adapters Cannot Guarantee

Both full-close branches of `decreasePosition` emit `PositionDecreased` with `collateralTotal`, `debtTotal`, and `currentLtv` hardcoded to zero and `healthFactorWad` hardcoded to `type(uint256).max`, in the [swap-free branch](#) and the [swap branch](#), instead of reading the resulting position as the [partial-decrease branch](#) does. This encodes the assumption that a full close leaves the account empty and infinitely healthy,

which holds only under the adapters' one position per `(owner, subId)` [usage requirement](#) that the router does not enforce. An account that co-locates a second position under the same `subId`, which is reachable through ordinary `increasePosition` calls, makes the emitted state incorrect.

For the Aave adapters the whole payload is unsound, because `AaveLendingAdapter.describePosition` and `AaveV4LendingAdapter.describePosition` read `currentLtv` and `healthFactorWad` from the account-level `getUserAccountData`, and closing one reserve neither repays another reserve's debt nor withdraws its collateral, so the account can remain liquidatable while the event reports it as closed and infinitely healthy. For the Compound adapter only `collateralTotal` is wrong, because `positionOf` reads `collateralBalanceOf` per asset and a second Comet collateral survives the close, whereas `borrowBalanceOf` is the account's single base borrow that the full repay clears, making the zero debt, zero LTV, and infinite health correct. The Morpho adapter is unaffected, because its `positionOf` reads an isolated per-market position that a full close zeroes exactly.

This residual collateral is not specific to Aave, because `IPool.supply`, the live Comet's `supplyTo`, and Morpho's `supplyCollateral` all accept an unauthorized `onBehalf`, so any party can add collateral to an account, though this only affects a full-close event when the supply lands between the position read and the withdrawal.

No funds are lost, but because the system ships an event-driven indexer, a position recorded as closed, empty, and perfectly healthy hides residual collateral and, for the Aave adapters, stops health monitoring on an account that still carries liquidation risk.

Consider reading the position in the full-close branch as the partial-decrease branch already does, or emitting a distinct full-close event without resulting-state fields, so that an all-zero and infinitely-healthy payload can never encode a closed position.

Update: Acknowledged, not resolved. The team stated:

Acknowledged for the Aave adapter only. A full close ends empty by construction for the documented one-position-per- `(owner, subId)` model, so the hardcoded zeros are correct there. The mismatch arises only when a user co-locates multiple Aave positions under one `subId`, which the docs explicitly warn against. The co-emitted `PositionUpdated` already carries a read-derived resulting state for indexers. We reinforce the one-position-per- `subId` guidance rather than add an oracle-dependent read to the close event (which L-05 shows can revert).

L-10 `ROUTE_SWAP` Leaves a Spendable Permit2 Allowance on the Router

The `_routeSwap` function of `MarginRouter` wraps a Universal Router call in a flash-take envelope. It takes up to `maxIn` of the input currency from the `PoolManager`, funds the Universal Router to pull what the swap costs, runs the caller-built route, and settles the unspent portion of the take. The funding step consists of an [unlimited ERC-20 approval to Permit2](#) followed by a [scoped Permit2 allowance to the Universal Router](#).

Neither approval is revoked once the route has run. The Permit2 allowance is written with an expiration of `block.timestamp`, and Permit2 rejects an allowance only when `block.timestamp` is strictly greater than the stored expiration, so the residual allowance remains spendable for the remainder of the block. On networks where consecutive blocks may share a timestamp, the window spans several blocks. Because the spender is the caller-supplied `universalRouter` address and a plan consisting of a single `ROUTE_SWAP` action nets to zero on its own, any account may plant such an allowance on the router for the cost of gas alone, naming an arbitrary spender, an arbitrary token, and an amount bounded only by the `PoolManager` balance of that token. No tokens, approvals, or open positions are required.

The router holds no funds between calls, so the residual allowance is not directly exploitable. However, it becomes exploitable during the window opened by `_routeSwap` itself. The [flash-take](#) precedes the route, and a v4 route settles its input only after the swap has executed, so the router holds the entire take while caller-supplied route code runs. Any party reachable from that route, e.g., a hook or a callback-bearing token, may spend a previously planted allowance at that moment. The subsequent [leftover computation](#) measures the balance rather than the amount the route consumed, so the shortfall is absorbed into the router's remaining debt and financed by the downstream `ACCOUNT_BORROW`. The loss therefore lands on the position owner as additional debt drawn without matching collateral. It is bounded by the unspent portion of `maxDebtIn`, which is a looser bound than the route's own `amountInMaximum`, and `maxLtvAfter` is optional on an increase and therefore may not detect it.

Consider revoking the Permit2 allowance immediately after the Universal Router call returns, by writing a zero amount for the same token and spender. Restricting the set of permitted Universal Router addresses would independently remove the ability to name an arbitrary spender, because a Universal Router only ever pulls funds from its own caller.

Update: Resolved at commit [a4d2d4a](#).

L-11 An Increase Supplies the Account's Entire Collateral Balance With No Way to Opt Out

The `equity` field of `IncreaseParams` determines how much collateral is pulled from the caller into the `MarginAccount` before the swap. Setting it to zero is supported and documented as the way to add leverage without contributing new equity. It does not, however, determine how much is supplied to the lending market. The increase [encodes its supply leg with the `OPEN\_DELTA` sentinel](#), which the `_supplyCollateral` function resolves to the account's entire balance of the collateral token.

Consequently, any collateral already held by the account is committed to the position, and no parameter can express the intent to supply only what the swap bought. An account that carries a balance for an unrelated purpose, for example, funds staged for a later repayment, or a token that serves as the collateral of one market and the debt of another under the same `subId`, has that balance absorbed by the next increase. Because the account address is derivable through `accountOf`, a third party can also cause this by transferring collateral to it. No funds are lost, since the balance lands in the owner's own position and remains withdrawable, but the position is altered beyond what was requested, the resulting loan-to-value moves accordingly, and the absorbed amount does not appear in the emitted event.

The behavior is furthermore inconsistent within the same contract. The `addCollateral` function supplies the explicit amount it pulled and leaves any pre-existing balance untouched, so two entry points apply opposite rules to the account's idle balance.

Consider recording the account's collateral balance before equity is pulled in, and encoding an explicit supply amount equal to the contributed equity plus `collateralToBuy` rather than the `OPEN_DELTA` sentinel. This shares a root cause and a remediation with the fill-assertion issue, so the two are best resolved together.

Update: Acknowledged, not resolved. The team stated:

Working as designed. The increase supplies `OPEN_DELTA` (the account's full collateral balance) so equity plus the bought collateral are committed atomically with no residual; this is the intended all-or-nothing construction. The M-03 fix makes the fill assertion measure the swap delta, so an idle balance is no longer able to mask a short fill. Integrators that want to keep a balance uncommitted use a distinct `subId`.

L-12 Full Close of a Debt-Free Empty Position Is a Silent No-Op

Passing `type(uint256).max` as `debtToRepay` to `decreasePosition` requests a full close. When the adapter reports no debt, the function takes the `swap-free branch` that withdraws the reported collateral directly to the caller. That branch is intended for a position funded only through `addCollateral`, repaid out of band, or previously liquidated.

The withdrawal is itself `guarded by a non-zero collateral balance`, so when both reads return zero the branch performs no state change. The call nonetheless returns successfully and `emits PositionDecreased` with a fully-closed payload: zero resulting collateral, zero resulting debt, a zero loan-to-value, and a maximal health factor. Because the function `returns only the account address`, a caller has no in-band means of distinguishing a completed close from one that did nothing.

The condition is reachable without any privileged action by supplying a `subId` or a registered `market` for which `the resolved account holds no position`. It is amplified when a registered pair is `re-pointed to a different underlying market`, since the reads then resolve against the new market and report an empty position while the original position remains open and liquidatable.

The practical harm falls on off-chain consumers. A de-risking process that closes a position on a health threshold receives a successful return and a canonical close event, marks the position closed, and stops monitoring it, while the position remains open. An indexer consuming the same event reaches the same conclusion.

Consider reverting with a named error when a full close is requested for a position that reports neither collateral nor debt. The case of no debt with non-zero collateral must continue to succeed, since that branch performs a genuine withdrawal.

Update: Acknowledged, not resolved. The team stated:

A full close of an already-empty position is idempotent: it withdraws any remaining collateral and emits `PositionDecreased/PositionUpdated` with the terminal (zero) state. It neither moves unexpected funds nor misreports, so we treat it as a harmless no-op rather than adding a revert that would complicate legitimate idempotent closes.

L-13 Re-Pointing a Registered Pair Orphans Every Position Already Opened Under It

The `MorphoLendingAdapter` resolves an agnostic `(collateral, debt)` pair to a concrete Morpho `MarketParams` through its `routing table`. Morpho Blue permits many markets for the same token pair, differing in oracle, interest rate model, and liquidation LTV, while `register` derives its storage key from the collateral and loan tokens alone. The `setMarket` function therefore overwrites an occupied slot without any occupancy check, and re-pointing a pair at a different Morpho market is the only curation lever the adapter offers, since it exposes no revocation path.

Once a pair is re-pointed, the market identifier derived from the stored parameters changes, so every read and every encode targets the new market while the account's balances remain in the old one. Consequently `positionOf` reports zero collateral and zero debt, `encodeRepay` encodes a repayment against a market in which the account holds no borrow shares, and `encodeWithdrawCollateral` encodes a withdrawal from a market in which it holds no collateral. Every position already opened under the pair becomes at once unobservable and unclosable through the router, while `isSupportedMarket` continues to report the pair as routable.

The failure does not surface as a revert. Because the debt reads zero, `decreasePosition` takes its swap-free full-close branch, and because the collateral also reads zero that branch withdraws nothing, yet the call returns successfully and emits a `PositionDecreased` event carrying a fully-closed payload. Every affected owner therefore receives a successful close that closed nothing. The positions are not permanently trapped, since `MarginAccount.repay` and `MarginAccount.withdrawCollateral` both accept a caller-chosen adapter and the owner-only `execute` accepts hand-built calldata. What is lost is the atomic self-funding unwind, since every fallback requires the owner to front external capital to repay before recovering collateral. The trigger is a routine governance action that affects every position in the pair simultaneously.

Consider rejecting a registration that would replace an occupied slot with a different market identifier, so that changing a pair's market requires an explicit revocation first and cannot silently orphan live positions. Keying the registry by market identifier rather than by token pair would achieve the same guarantee structurally.

Update: Acknowledged, not resolved. The team stated:

Confirmed; the counterpart of L-02. Re-pointing a pair switches the registry to a new market, so reads/encoders no longer see positions opened under the old one (a full

close reads zero and is a success-shaped no-op). Positions remain recoverable via `MarginAccount.execute` with the old `MarketParams`. As a governance-only action, we document "do not re-point a pair with open positions" rather than add registry versioning.

L-14 Repay Amount Clamping Is Inconsistent Across Adapters and Misreported

Nothing on the partial-decrease path bounds `debtToRepay` against the account's outstanding debt, so an over-sized amount reaches the lending protocol unchanged. Three of the four adapters absorb it by clamping. The `Compound adapter` caps the amount at `borrowBalanceOf` before encoding the call. The `Aave v3 adapter` forwards the amount to `IPool.repay`, which reduces it to the user's debt internally. The `Aave v4 adapter` forwards it to the Spoke, whose cap at the total debt is recorded in [the vendored interface](#). The `Morpho adapter`, however, forwards the amount as assets, and Morpho converts it to shares and subtracts them from the account's `borrowShares`. That subtraction underflows, so the call reverts with an arithmetic panic rather than clamping.

Where the amount is clamped, the router reports the requested figure rather than the performed one. The `PositionDecreased event` emits `params.debtToRepay`, even though `IMarginAccount.repay` already returns the repaid amount measured as the debt-token balance decrease. A decrease sized above the outstanding debt therefore reports a repayment that did not occur.

The surplus debt tokens the route over-bought also remain in the `MarginAccount`, because [the sweep that returns it](#) runs only on the full-close branch. The surplus itself is recoverable by the owner through `MarginAccount.sweep`. The cost of acquiring it is not: [the fill check](#) requires the account to hold the full requested amount, so the route must buy debt the position cannot use, and the swap fees and slippage on that excess are lost.

Because `ILendingAdapter` does not specify the expected behavior for an over-sized repay, an integrator cannot predict which outcome applies. The same router call succeeds with an inaccurate event on three adapters and reverts with an undiagnosable panic from inside the `PoolManager` unlock on the fourth.

Consider bounding `debtToRepay` against the outstanding debt on the partial-decrease path, which would make the event accurate by construction, leave no surplus to sweep, avoid the unnecessary purchase, and remove the Morpho revert. It is also advisable to emit the value returned by `repay` rather than the requested amount, and to run the surplus sweep on both

decrease branches, so that any amount reaching an adapter unbounded is still reported and returned correctly. Finally, consider specifying the expected clamping behavior in `ILendingAdapter` and aligning the Morpho adapter with it. The full-repay branch of that adapter already reads `borrowShares` and could cap the asset-denominated path in the same way.

Update: Partially Resolved at commit [16319b9](#). The team stated:

The partial-decrease `PositionDecreased` reports the requested `debtToRepay`; where a venue clamps, the true repaid amount can differ. No funds are at risk: the surplus is swept on a full close, and a partial decrease that requests at most the live debt repays exactly that amount. A request above the live debt clamps on every venue (Compound in-encoder, Aave protocol-side, Morpho at the reported debt after the L-01 fix) and the event then over-reports, with the co-emitted `PositionUpdated` carrying the true resulting debt (best-effort, per L-05). Threading the measured repaid amount up from inside the unlock is disproportionate to the impact; the field's requested-amount semantics are documented in the event NatSpec and the events reference.

L-15 A Drifted Compound V3 Position Rejects Small Partial Decreases

Compound v3 enforces two collateral factors per asset. Comet re-checks the borrow collateral factor through `isBorrowCollateralized` on every collateral withdrawal, while the higher liquidate collateral factor marks the liquidation boundary. A position whose loan-to-value has drifted into the band between them is therefore not yet liquidatable, yet cannot freely release collateral. The `maxLtvWad` function reports only the liquidate factor, so the borrow factor never reaches `PositionData` and remains invisible to an integrator.

A partial `decreasePosition` repays the requested debt and then withdraws the collateral the swap consumed, resolved through [the `OPEN\_DELTA` sentinel](#). Because the repayment precedes the withdrawal, Comet evaluates the borrow factor against the reduced debt. Writing `b` for the borrow collateral factor, `d` for the debt value, and `c` for the collateral value, a decrease that repays value `D` and withdraws value `W` succeeds only when $d - D \leq b * (c - W)$. A drifted position therefore has a minimum viable decrease size, and any smaller request reverts. At a borrow factor of 0.80, a liquidate factor of 0.85, and a loan-to-value of 0.82, that minimum is approximately 12% of the outstanding debt, and swap costs raise it further by making `W` exceed `D`. The router neither computes nor surfaces this threshold, so a user requesting a smaller decrease receives an opaque Comet revert from inside the `PoolManager` unlock. A full close is unaffected, as it clears the debt before withdrawing.

Consider exposing the borrow collateral factor through `PositionData`, so that an integrator can compute the minimum viable decrease before submitting a transaction that cannot succeed. It is also advisable to document the constraint on the decrease path. Consider additionally offering a repay-only decrease mode funded by the caller rather than by sold collateral, which reduces risk without releasing any collateral.

Update: Partially Resolved at commit [16319b9](#). The team stated:

Confirmed as inherent Compound v3 behavior. Comet re-checks the borrow collateral factor on the repay-before-withdraw ordering, imposing a minimum viable decrease; the adapter surfaces only the liquidate factor via `maxLtvWad`. This is a disclosure/UX matter, not a defect: a full close always works, and callers can size decreases accordingly. The minimum-decrease behavior (size against the borrow collateral factor, not `maxLtvWad`) is documented for integrators in [16319b9](#).

L-16 CompoundV3LendingAdapter Values Debt With a Superseded Comet Price Feed

The `CompoundV3LendingAdapter` contract derives its risk figures from Comet's price feeds in the `_positionValues` function. The collateral feed is re-read on every call through `getAssetInfoByAddress`. The base feed, however, is read once into an immutable in the constructor, on the documented grounds that it is immutable per Comet instance in the same way as `baseToken`.

That premise does not hold. A Comet sits behind a proxy and its price feeds are immutables of the implementation, so Compound governance changes the base feed by calling `Configurator.setBaseTokenPriceFeed` and then `deployAndUpgradeTo`, leaving the proxy address unchanged. Compound's `Configurator` contract treats the two fields as opposites: `baseToken` and `trackingIndexScale` are the only fields that `setConfiguration` refuses to modify, reverting with `ConfigurationAlreadyExists`, while `baseTokenPriceFeed` has a dedicated governance setter. Furthermore, Comet's `getPrice` function accepts any address without checking that it is a registered feed. Therefore the adapter continues to price debt with a feed the Comet has abandoned, and no observable change to the Comet address signals that this has occurred.

This has already occurred on live markets. Transaction [0x494079fa](#) executed Compound proposal 338 on 7 October 2024, calling `setBaseTokenPriceFeed` and `deployAndUpgradeTo` on the mainnet `cWstETHv3` market and emitting `Upgraded` on the unchanged proxy at `0x3D0bb1ccaB520A66e607822fC55BC921738fAFE3`. That upgrade

replaced a live wstETH-in-ETH feed with a constant feed returning exactly `1e8`. The superseded feed still prices wstETH collateral in the `cWETHv3` market, where it currently reports `124092181`. [Proposal 539](#) later did the same to the `cWRONv3` base feed on Ronin, after Chainlink discontinued support for that chain.

The resulting `debtValue` is the denominator of both `currentLtvWad` and `healthFactorWad`, so it determines what `describePosition` reports and whether the `ASSERT_HEALTH` action in the `MarginRouter` contract passes. The divergence from Comet's own valuation is unbounded, and its direction is determined by the feed Compound governance selects. Neither direction is acceptable. Where the superseded feed reads above the current one, debt is overvalued and health is reported as worse than Comet considers it, so the mandatory partial-decrease bound rejects positions that are in fact within it. An adapter deployed before proposal 338 would value wstETH debt approximately 24% above Comet's own valuation, and the gap widens permanently. Where the superseded feed reads below the current one, health is reported as better than Comet considers it, and `ASSERT_HEALTH` accepts a position that Comet already treats as liquidatable. This removes the only protection a caller supplying `maxLtvAfter` receives.

If a superseded feed is retired rather than left running, `getPrice` reverts with `BadPrice` and every priced read reverts with it. The `increasePosition` and `addCollateral` functions both fail on their trailing `describePosition` call, and a partial `decreasePosition` fails on `ASSERT_HEALTH`, which cannot be skipped because a non-zero `maxLtvAfter` is mandatory on that path. Among the curated entry points, only a full close remains available, although `execute` plans continue to function because the position snapshot is emitted inside a `try/catch` block. Because the cached value is immutable, recovery requires deploying a replacement adapter and re-registering every market.

Consider reading `comet.baseTokenPriceFeed()` at the point of use, mirroring the treatment already applied to the collateral feed. The `baseToken` and `baseScale` values may remain cached, as `Configurator` rejects any change to the base token. Consider also correcting the docstring on the cached feed, which cites `baseToken` as the precedent for a field that Compound governance treats as freely settable.

Update: Resolved at commit [8b914aa](#).

Notes & Additional Information

N-01 Misconfigured Morpho Markets Can Be Registered as Routable

The `setMarket` function of `MorphoLendingAdapter` gates registration on [a single check](#), namely that the market's loan token is not the zero address. Morpho Blue's `createMarket` carries no access control and constrains only the interest rate model and the liquidation LTV, placing no constraint on the token or oracle addresses. A market with a zero collateral token, a zero oracle, the same address on both sides, or simply the wrong oracle therefore passes registration.

The zero-collateral-token case is not hypothetical. Morpho Blue reserves that configuration for idle markets, in which the loan token is real while the collateral token, the oracle, and the interest rate model are all the zero address and the liquidation LTV is zero. The live mainnet deployment enables both prerequisites, so any account may create such a market, and the idle markets for USDC, WETH, and USDT are already active and hold deposits. Each carries a non-zero loan token and therefore satisfies the adapter's only validation. Registering one would store it under a key whose collateral is the native-currency sentinel while the oracle address is zero.

The consequences are not uniformly benign. A zero collateral token or a zero oracle yields a route that reverts. A market naming the same token on both sides is enterable, since supplying collateral succeeds and no check compares the two currencies. In every case

`isSupportedMarket` reports the pair as routable to off-chain consumers. Separately, the documentation of `resolve` states that a registered market always has non-zero collateral and loan tokens, an invariant that is not enforced. The condition that actually makes the both-zero sentinel sound is narrower, namely that every entry carries a non-zero loan token, and it resides in `setMarket` rather than in the registry.

Consider validating in `setMarket` that the collateral token and the oracle are not the zero address and that the collateral and loan tokens differ.

Update: Acknowledged, not resolved. The team stated:

Acknowledged, with a wording correction. `setMarket` gates on `morpho.idToMarketParams(id).loanToken != 0` (a Morpho-existence check), not a bare `loanToken != 0` as described. An idle-but-real Morpho market passes, so a nonsensical pair can be registered and would break the `resolve` doc-invariant, but every downstream failure is an atomic revert or a same-token no-op market, and only governance can register markets which reasonably bounds the practical impact.

N-02 Morpho Position Views Revert on Reads the Result Does Not Depend On

The `_positionValues` helper of `MorphoLendingAdapter` backs both `currentLtvWad` and `describePosition`, and `positionOf` performs part of the same work. [Two external reads](#) are made unconditionally, neither of which affects the result for a debt-free position.

The first is the market oracle. `IOracle.price()` is called even when the account carries no debt, in which case the loan-to-value is definitionally zero and the health factor definitionally maximal. The second is the accrued debt. `MorphoBalancesLib.expectedBorrowAssets` accrues interest through the market's interest rate model even when the account's borrow-share balance is zero, so an interest rate model that reverts under a `STATICCALL` makes the read fail for an account whose debt is provably zero.

Either failure propagates into router flows that read positions without a guard. The practical instance is `addCollateral`, which reads `describePosition` purely to populate its event, so a first deposit into a fresh account cannot complete while the oracle or the interest rate model is reverting, even though Morpho Blue's own `supplyCollateral` requires no price. For a position that does carry debt both reads are genuinely required, and every adapter in the suite reads its protocol's price source the same way, so only the debt-free case is avoidable. Separately, `ILendingAdapter` does not state that these reads depend on the protocol's price source and interest rate model and may therefore revert.

Consider reading the account's borrow-share balance first and short-circuiting both reads when it is zero, as the full-repay branch of `encodeRepay` already does.

Update: Partially Resolved at commit [3d03ca5](#). The team stated:

Partially fixed in `3d03ca5`; remainder acknowledged. The harmful case, `addCollateral`'s trailing `describePosition` letting a reverting oracle block a first deposit, is resolved: that curated event read is now wrapped in try/catch (the L-05 fix), so a risk-reducing deposit completes even when the venue oracle is down. The underlying observation that `_positionValues` consults `oracle.price()` and the

accruing `expectedBorrowAssets` even for a debt-free position is inherent to reusing Morpho's own libraries; it affects only view reads and we accept it as-is.

N-03 Pending Ownership Handoffs Cannot Be Canceled

The `propose` function of the `Owner` type rejects the zero address, so once a successor has been proposed through `transferOwnership`, the pending slot can never be cleared and can only be overwritten with another non-zero address. As a consequence, a mistaken or abandoned proposal remains acceptable indefinitely, and since no event is emitted when a handoff is proposed, the existence of a live proposal is not surfaced to off-chain monitoring. In contrast, OpenZeppelin's `Ownable2Step` accepts `transferOwnership(address(0))` precisely as the path to cancel a pending transfer, while its two-step design still prevents the role from being handed off to an unrecoverable address.

Consider allowing the zero address in `propose` so that a pending handoff can be explicitly canceled, or alternatively documenting that the practical way to neutralize a proposal is for the owner to propose their own address.

Update: Partially Resolved at commit [c681ca4](#). The team stated:

Acknowledged, with a correction. `Owner.propose` rejects the zero address and only `acceptOwnership` clears the pending owner, so a proposal can be overwritten but not neutralized, and the current owner keeps full power until acceptance, so there is no window of lost control. Partially addressed in [c681ca4](#), which adds proposal and handoff events; the router's `transferGovernance` already emits `GovernanceTransferStarted`. Rejecting zero is deliberate, to avoid bricking ownership.

N-04 Shadowed State Variable

The `MarginAccountFactory` contract declares an `accountImplementation` state variable. The constructor of `MarginRouter`, which inherits from that factory, declares a parameter of the same name, so within the constructor the identifier resolves to the parameter rather than to the inherited state variable. Every other parameter of that constructor follows the trailing-underscore convention, as does the factory's own constructor parameter.

Consider renaming variables that shadow any state variable, in order to avoid undesirable behavior and improve code clarity.

Update: Resolved at commit [1734a1e](#).

N-05 CollateralWithdrawn Always Reports a Zero Amount on Three of the Four Venues

The `withdrawCollateral` function of `MarginAccount` measures the withdrawn amount as the increase in the account's own collateral balance across the call, and emits that measurement. Aave v3, Compound v3, and Morpho Blue all deliver the withdrawal directly to the receiver rather than to the account, so the measured delta is zero and `CollateralWithdrawn` reports `0` for every withdrawal on those three venues. Only the Aave v4 adapter, whose encoder delivers to `msg.sender`, produces a non-zero value. The behavior is acknowledged in the function's `@dev` note, but the event remains unusable for off-chain accounting on three of the four supported venues.

Consider emitting the requested amount in addition to the measured delta, so that the event carries a meaningful figure for every adapter.

Update: Acknowledged, not resolved. The team stated:

Acknowledged. `MarginAccount` logs the measured account-balance delta, and the Morpho, Aave v3, and Compound encoders deliver the withdrawal straight to the receiver (account delta zero), so only Aave v4 (which routes through the account) logs a non-zero amount. The transfer is correct; only the logged amount is a false zero, and it is the escape-hatch withdrawal event, so indexers undercount owner-driven withdrawals. Not a fund-safety issue; the field's per-venue meaning is documented in the event `NatSpec` and the events reference ([16319b9](#)).

N-06 Unused Named Return Variables

Named return variables are a way to declare variables that are meant to be used within a function's body for the purpose of being returned as that function's output. They are an alternative to explicit in-line `return` statements.

Every function in `MarginCallldataDecoder.sol` declares named return variables and then returns the result of `abi.decode` through an explicit `return` statement, so none of the twenty-four declared names is ever assigned. The affected functions are `decodeAdapterMarketAmount`, `decodeAdapterMarketAmountReceiver`, `decodeSweep`, `decodeHealthCheck`, `decodeFillCheck`, `decodeSubId`, `decodePull`, and `decodeRouteSwap`.

Consider either using or removing any unused named return variables.

Update: Resolved at commit [1734a1e](#).

N-07 Unused Code

Throughout the codebase, multiple declarations are never referenced:

- The `Position` struct imported from Morpho Blue's `IMorpho.sol` in `MorphoLendingAdapter.sol`.
- The `Currency` type imported in `MorphoLendingAdapter.sol`.
- The `raw` and `lte` functions of `Ltv.sol`, both attached to the `Ltv` type by a `using` directive but never invoked. Only `gt` is used.

Consider removing the above instances to improve the overall clarity and readability of the codebase.

Update: Resolved at commit [1734a1e](#).

N-08 Inconsistent Interface Convention in Storage Types

The `Owner.write` and `Owner.propose` functions return the `Owner storage` reference to allow call chaining, whereas `Owner.acceptOwnership` returns nothing.

The same split exists between the sibling market-table types: `MarketRegistry.register` returns the `MarketRegistry storage` reference for chaining, whereas `MarketAllowlist.set` returns nothing, even though `MarketAllowlist` is documented as following the same type-driven shape as `MarketRegistry`. In all of the above cases, the returned storage reference is discarded at every call site.

Consider adopting a single convention for these storage types, either by having all state-mutating functions return the storage reference or by removing the unused return values, in order to improve the consistency and readability of the codebase.

Update: Resolved at commit [1734a1e](#).

N-09 An Unsupported Action Opcode Is Reported as a Missing Active Account

The action dispatcher of `MarginRouter` resolves opcodes in three ranges. Opcodes at or above the account-scoped range fall through to the `active-account guard`, and only afterwards reach the `trailing UnsupportedAction revert`.

Because the guard precedes opcode validation, a plan carrying an undefined opcode above `ASSERT_ACCOUNT_BALANCE` reverts with `NoActiveAccount` rather than `UnsupportedAction` whenever no account has been bound. The reported error therefore describes a condition that is not the cause, which misleads a plan builder debugging an encoding mistake into adding a `SET_ACCOUNT` action that cannot help.

Consider validating the opcode before consulting the active-account guard so that an unknown opcode is always reported as such.

Update: Resolved at commit [36f9493](#).

N-10 Zero Is Overloaded as a Control Signal in Amount and Health-Bound Fields

The margin action interpreter reserves specific values of ordinary fields as control signals. `ActionConstants.OPEN_DELTA` is `0`, and the health assertion treats a zero `Ltv` as an instruction to skip the check. Both are sound where the router encodes the sentinel deliberately. Neither is sound where a computed value can coincide with the sentinel, and two such cases exist:

- `decreasePosition` encodes `collateralBefore` as the full-close withdrawal amount. A position carrying debt with no collateral in the market therefore encodes `0`, which `_withdrawCollateral` resolves to `_getFullDebt(market.collateral)`, namely the collateral the close swap consumed. The account is then asked to withdraw that amount from a position holding none. The condition is reachable on both Aave adapters, whose `positionOf` reads a single collateral reserve, either through a bad-debt liquidation or through an account that holds a second collateral under the same `subId` and withdraws the first.
- `_assertHealth` treats a zero bound as a request to skip the assertion, so a caller cannot express a resulting loan-to-value of zero, that is, a requirement that no debt remain. On an increase such a bound is silently ignored, and on a partial decrease it is rejected as `SlippageBoundRequired`.

Consider carrying these signals out of band rather than encoding them in the value field, for example through an explicit flag or a distinct opcode.

Update: Partially Resolved at commit [16319b9](#). The team stated:

Acknowledged. Two low-consequence cases: a with-debt, zero-collateral full close encodes `0`, which collides with the `OPEN_DELTA` sentinel and produces an opaque venue revert; and a zero LTV bound is unexpressible (treated as skip). Both are revert-or-skip with no fund impact; the sentinel semantics (the inexpressible debt-with-zero-collateral close and the zero-LTV skip bound) are documented on `DecreaseParams` in [16319b9](#).

N-11 Sweeping Native Currency From an Account to the Manager Always Reverts

The `ACCOUNT_SWEEP` action forwards a token balance out of the active account, and the `sweep` function of `MarginAccount` accepts either the owner or the manager as the recipient. A `MarginAccount` can hold native currency, since it declares a permissive `receive` function for adapter calls that require value.

However, the manager recipient is unusable for native currency. The `receive` function inherited by `MarginRouter` rejects any sender other than WETH9 and the `PoolManager`, so a sweep of native currency to the router reverts with `InvalidEthSender`. Funds are not trapped, because a sweep naming the owner succeeds, but one of the two documented recipients does not apply to one of the supported currencies.

Consider documenting that native currency must be swept to the owner, so that a plan builder does not encode the manager recipient for it.

Update: Acknowledged, not resolved. The team stated:

Acknowledged. A native-currency account sweep to the manager reverts because the router's `receive()` guard rejects unsolicited native value; an owner recipient works, so funds are never trapped. The path is unreachable in-tree today (every curated adapter uses `value = 0`); we note the manager-recipient native-sweep limitation.

N-12 Functions Updating State Without Event Emissions

Throughout the codebase, multiple instances of functions updating the state without an event emission were identified:

- The `constructor`, `transferOwnership`, and `acceptOwnership` functions of `OwnableAdapter.sol`. The contract declares no events at all, even though each of the three writes the owner slot.
- The `constructor` function of `MarginRouter.sol`, which establishes the governance address. The router emits an event for both steps of a later governance handoff, so only the initial assignment is unobservable.

The adapter case is the most consequential, because the owner of `OwnableAdapter` is the only account permitted to curate that adapter's market routing, that is, which `(collateral, debt)` pairs it serves and which concrete lending-protocol market each pair resolves to. Neither the proposal of a successor nor the completion of a handoff leaves an on-chain record, so off-chain monitoring must poll `owner` and `pendingOwner` to detect that the account controlling the routing table has changed. This is inconsistent with the rest of the codebase, since `MarginRouter` composes the same `Owner` type for its governance role and emits an event for both steps, and every adapter emits `MarketSet` whenever its routing is curated. The indexer included in this change set subscribes to `MarketSet` and `AdapterAllowed` but to no ownership event, so it records every curation action without being able to attribute it to the role holder authorized to perform it.

Consider emitting events whenever there are state changes, including on the initial assignment performed in a constructor, mirroring OpenZeppelin's `Ownable`, which emits `OwnershipTransferred` from its own constructor.

Update: Resolved at commit [c681ca4](#).

N-13 A Comet Base Supply Position Is Not Reachable Through the Router

Comet converts any base token supplied beyond the outstanding borrow into a positive base balance. For such an account `borrowBalanceOf` returns zero, so `positionOf` reports no debt and the `debt-free full-close branch` withdraws only the collateral, leaving the base supply in the market. The balance cannot be recovered through `ACCOUNT_WITHDRAW_COLLATERAL` either, since `setMarket` rejects a pair whose collateral is the Comet's base token. Recovery

therefore requires the owner-only `MarginAccount.execute` with a hand-built `IComet.withdraw` call. Such a base supply cannot be created through the router's own flows, because `encodeRepay` clamps every repayment to the outstanding borrow, so it arises only through the owner escape hatch or a direct interaction with Comet.

Consider documenting that a base supply position falls outside the router's reach, or exposing a base-token withdrawal path that does not require a registered collateral market.

Update: Partially Resolved at commit [16319b9](#). The team stated:

Acknowledged (doc gap). A Comet base-asset supply has `borrowBalanceOf == 0`, so a router close withdraws only collateral, and the base asset cannot be registered as collateral (`getAssetInfoByAddress(base)` reverts to `MarketNotSupported`). A base supply arises only via `execute` or a direct Comet call and is recoverable via `MarginAccount.execute`; the router's base-supply exclusion is documented in the Compound venue notes in [16319b9](#).

N-14 Priced Reads Revert if a Comet Collateral Asset Is Removed

Comet's `getAssetInfoByAddress` reverts for an asset that is not a registered collateral asset of the market. The `setMarket` function accounts for this and wraps the call in a `try / catch`, treating a revert as an unsupported pair. The priced read paths do not: `maxLtvWad` and `_positionValues` call it directly, and both `currentLtvWad` and `describePosition` resolve through `_positionValues`. Reads for a market whose collateral asset has been removed therefore revert opaquely from inside the `PoolManager` unlock instead of reporting the market as unsupported.

The impact is bounded by Compound's own design. `Configurator` exposes no asset-removal function, so a removal requires `setConfiguration` to replace the entire configuration with a shorter asset array. Furthermore, `numAssets` is an implementation `immutable` that `isBorrowCollateralized` iterates, so a removed asset stops counting toward collateral value, `withdrawCollateral` reverts for it, and `absorb` can no longer seize it. Removing an asset that still holds balances would therefore strand that collateral and undercollateralize every position holding it. The realistic case is consequently a removal performed only after balances reach zero, in which case no position is affected. No adapter change can restore access to a removed asset, since the withdrawal reverts inside Comet.

Consider mirroring the `try / catch` treatment from `setMarket` in the priced read paths, so that an affected market reports itself as unsupported rather than reverting opaquely.

Update: Partially Resolved at commit [16319b9](#). The team stated:

Acknowledged (robustness). `setMarket` guards `getAssetInfoByAddress` with `try/catch`, but `maxLtvWad` and `_positionValues` call it unguarded, so removing a Comet collateral asset makes those reads revert opaquely. Bounded by Compound's design (a collateral is realistically removed only after balances are zeroed), and a reverting read does not risk funds; the dependence on the live Comet asset set is documented in the Compound venue notes in [16319b9](#).

N-15 Incomplete and Inaccurate Documentation

Throughout the codebase, multiple documented statements do not hold:

- The [positionOf specification](#) and [PositionData.collateralAmount](#) state that collateral is returned with accrued interest applied. This holds for the [Aave adapters](#) but not for [Morpho Blue](#) or [Compound v3](#), whose collateral does not accrue.
- [describePosition](#) states that the health factor is `maxLtv / currentLtv`. That identity is exact only for the adapters that compute it from the same factor they report as `maxLtv`, namely [Morpho](#) and [Compound v3](#). The Aave adapters ([v3](#), [v4](#)) pass the protocol's own health factor through against a differently sourced `maxLtv`, so the relationship is approximate there.
- The [cross-collateral caveat](#) on [describePosition](#) names only the Aave adapters, omitting [Compound v3](#), whose base debt read is account-level while its collateral read is per-asset. [currentLtvWad](#) carries no caveat at all, even though it is the function the [router's health assertion](#) consumes.
- [PositionData.healthFactorWad](#) and [describePosition](#) document the `type(uint256).max` case for a debt-free position but omit the zero case, i.e., debt held against no collateral value, which [MorphoLendingAdapter](#) documents correctly.
- The [MorphoLendingAdapter header](#) states that no Morpho math is reimplemented. The [collateral valuation](#) and the [loan-to-value](#) and [health-factor](#) computations are reimplemented, using `Math.mulDiv` in place of Morpho's checked `mulDivDown`, so the adapter returns values in numeric ranges where Morpho Blue's own health check reverts.
- The [AaveLendingAdapter](#), [AaveV4LendingAdapter](#), and [CompoundV3LendingAdapter](#) headers each state that no protocol math is reimplemented, the same claim described above for Morpho. All three reimplement the current-LTV computation locally ([Aave v3](#), [Aave v4](#), [Compound v3](#)), and Compound additionally reimplements [collateral valuation](#) and the [health factor](#) with `Math.mulDiv`, since none of the Pool, the Spoke, or the Comet exposes an LTV read.

- The [encodeRepay specification](#) describes the full repay as share-based, burning the entire borrow-share balance. That mechanism is [Morpho-specific](#): Aave [repays natively](#) and Compound, which has no shares, [caps an asset amount](#). The router repeats the framing in [_repay](#).
- The [PositionUnhealthy error](#) is documented as thrown when a position exceeds the adapter's maximum liquidation LTV. The only [throw site](#) compares against the caller-supplied bound and performs no adapter-LLTV check, and a zero bound skips the check entirely.
- The [execute events note](#) states that plans emit no [Position*](#) snapshot events. Every account-scoped opcode now emits [PositionUpdated](#), which the event's own [documentation](#) confirms fires inside [execute](#).
- The [IncompleteFill error](#) is documented only for an [increasePosition](#) collateral under-fill. It is also thrown by [ASSERT_FILL](#) for any currency against router credit, and by [decreasePosition](#) against the debt token (via [_assertAccountBalance](#)).
- The [minHopPriceX36](#) struct fields ([exact-input single](#), [exact-input](#), [exact-output single](#), [exact-output](#)) carry no NatSpec, so the [X36 scale](#) and [output-per-input orientation](#) are documented nowhere. The [per-hop error descriptions](#) say the exact-output swap "exceeds" its limit, while the code [reverts when the realized price falls below the minimum](#).
- The [MarginAccountFactory header](#) states that accounts can only be deployed and driven by the inheriting router. The account owner drives it directly through [execute](#) and the [manager-or-owner auth](#), bypassing the router's allowlist, and [deployment is permissionless](#); [IMarginRouter.createAccount](#) similarly calls it "the caller's" account.

Consider correcting each of the above statements so that the documentation describes the behavior of every implementation it covers, and thoroughly documenting all functions that are part of a contract's public API. When writing docstrings, consider following the [Ethereum Natural Specification Format](#) (NatSpec).

Update: Resolved at commit [16319b9](#).

N-16 Market Registration Validates the Wrong Reserve Property on Both Sides of the Pair

The [AaveLendingAdapter.setMarket](#) only checks that each asset has a non-zero `aToken`, meaning that it is a listed reserve. It never checks that the collateral side is usable as collateral or that the debt side is borrowable, and it does not require the two currencies to

differ. All the needed fields (`ltv`, `liquidationThreshold`, `usageAsCollateralEnabled`, `borrowingEnabled`, `isActive`, `isFrozen`) come from the same `getReserveConfigurationData` view the adapter already uses in `maxLtvWad`, which `setMarket` would call once per side.

As a result, an allowlisted pair can be permanently unopenable. A collateral with `ltv == 0` (for example DAI, LINK, USDe, PYUSD, USDS on mainnet, each a listed reserve with `usageAsCollateralEnabled == true` yet `ltv == 0`) is never auto-enabled as collateral on supply, because Aave's auto-enable requires `ltv != 0`, so the account's weighted LTV stays zero and the borrow reverts `LtvValidationFailed` (the current mainnet Pool has no earlier `CollateralBalanceIsZero` guard on the borrow path). A debt with `borrowingEnabled == false` (for example wstETH) reverts `BorrowingNotEnabled`, and a frozen reserve reverts `ReserveFrozen` on supply. Yet `isSupportedMarket` still reports the pair as routable, and the only observable failure is an opaque Aave revert from inside the `PoolManager` unlock. The `MarketSet` event carries just the two currencies and a bool, so the misconfiguration cannot be vetted off-chain either.

Of the sibling adapters, only `CompoundV3LendingAdapter.setMarket` validates the role of each side, requiring the debt to equal the Comet base token and the collateral to be a registered Comet collateral asset. `AaveV4LendingAdapter.setMarket` only checks that each reserve's `underlying` matches its currency and that both reserves share a Hub, which is identity validation rather than role validation, so it carries the same gap as this adapter.

Consider validating in `setMarket` that the collateral reserve is active, unfrozen, and has non-zero `ltv` and `liquidationThreshold` with `usageAsCollateralEnabled`. The debt reserve should be active, unfrozen, and `borrowingEnabled`, and the two currencies should differ. Optionally emit the validated risk parameters in `MarketSet` for off-chain review.

Update: Acknowledged, not resolved. The team stated:

Acknowledged (related to L-03). Confirmed: the Aave v3 `setMarket` reads `getReserveTokensAddresses(...)[0]` (the `aToken`) on both sides, which is role-inappropriate on the debt side (should check borrowing/variable-debt) and insufficient on the collateral side (should check `usageAsCollateralEnabled / ltv`), so `isSupportedMarket` can over-report. There is no fund-loss or stranding path: `increasePosition` is atomic and reverts wholesale on a bad market, `addCollateral` funds stay withdrawable, and use-time valuation reads the role-correct tokens. The realized consequence is a governance-gated unopenable market / opaque revert / misleading `isSupportedMarket`; the title overstates the impact.

Conclusion

The margin trading periphery assembles leveraged spot exposure in a single transaction by composing a Uniswap v4 swap with a borrow against an external lending venue, and it holds each position in a per user account that acts as its own supplier and borrower. OpenZeppelin audited the router, the account and its factory, the four lending adapters, and the supporting action, decoder, and value type libraries.

Responsibilities are separated across those layers. The router orchestrates flows and never holds a position, the account is the lending counterparty and constrains who may move its funds, and the adapters only encode calls and read state. Account authority is derived from the authenticated caller rather than from calldata, and the curated flows are guarded by explicit slippage and health bounds.

The findings cluster in the seams between those layers rather than in any one of them. The most consequential theme is the coupling to the integrated lending venues: adapters bind to venue components at deployment and are neither upgradeable nor equipped with setters, so a price feed replacement, a reserve reconfiguration, or an implementation upgrade at Morpho, Aave, or Compound can silently change what the router reports or prevent a flow from completing, in some cases including the path used to reduce risk. A second theme concerns the guarantees the action surface actually provides, where a fill assertion measures an absolute balance rather than the amount a swap delivered, a sentinel amount carries different meanings across neighbouring actions, and a position swap dispatches to a caller-supplied address. A third theme is the market curation lifecycle, where registering, replacing, or retiring a routable pair affects positions already open under it, and validation at registration time differs by venue. A fourth theme is event fidelity, which matters more than usual because this change set ships an event-driven indexer: several flows emit a resulting state the position does not hold, so a consumer can record a position as closed and healthy while collateral or debt remains.

No issue permits one user to reach another user's account or funds, and the isolation properties that guarantee this held throughout the review. The recurring pattern instead is that a failure surfaces as a successful transaction rather than a revert, which places the burden of detection on the off-chain monitoring that the same events are meant to feed. Reads that only populate event payloads are treated as load-bearing in the curated flows while the same reads are best-effort elsewhere, and tightening that asymmetry, together with re-deriving venue

bindings at the point of use, would remove a large share of the reported surface. The `execute` entry point deliberately enforces none of the curated guardrails and its safe composition rules are documented rather than enforced, so integrator-facing documentation is part of this system's security boundary and warrants the same review as the code. Test coverage is extensive across unit, integration, invariant, and fork suites, and extending the fork suites to cover venue reconfiguration and upgrade scenarios would exercise the conditions behind the most consequential findings.

We are grateful to the Uniswap team for their collaboration and responsiveness throughout this engagement.

Appendix

Issue Classification

OpenZeppelin classifies smart contract vulnerabilities on a 5-level scale:

- Critical
- High
- Medium
- Low
- Note/Information

Critical Severity

This classification is applied when the issue's impact is catastrophic, threatening extensive damage to the client's reputation and/or causing severe financial loss to the client or users. The likelihood of exploitation can be high, warranting a swift response. Critical issues typically involve significant risks such as the permanent loss or locking of a large volume of users' sensitive assets or the failure of core system functionalities without viable mitigations. These issues demand immediate attention due to their potential to compromise system integrity or user trust significantly.

High Severity

These issues are characterized by the potential to substantially impact the client's reputation and/or result in considerable financial losses. The likelihood of exploitation is significant, warranting a swift response. Such issues might include temporary loss or locking of a significant number of users' sensitive assets or disruptions to critical system functionalities, albeit with potential, yet limited, mitigations available. The emphasis is on the significant but not always catastrophic effects on system operation or asset security, necessitating prompt and effective remediation.

Medium Severity

Issues classified as being of medium severity can lead to a noticeable negative impact on the client's reputation and/or moderate financial losses. Such issues, if left unattended, have a moderate likelihood of being exploited or may cause unwanted side effects in the system.

These issues are typically confined to a smaller subset of users' sensitive assets or might involve deviations from the specified system design that, while not directly financial in nature, compromise system integrity or user experience. The focus here is on issues that pose a real but contained risk, warranting timely attention to prevent escalation.

Low Severity

Low-severity issues are those that have a low impact on the client's operations and/or reputation. These issues may represent minor risks or inefficiencies to the client's specific business model. They are identified as areas for improvement that, while not urgent, could enhance the security and quality of the codebase if addressed.

Notes & Additional Information Severity

This category is reserved for issues that, despite having a minimal impact, are still important to resolve. Addressing these issues contributes to the overall security posture and code quality improvement but does not require immediate action. It reflects a commitment to maintaining high standards and continuous improvement, even in areas that do not pose immediate risks.